# GPU Architecture Primer

## What Makes a GPU Different from a CPU

A CPU is optimized for low-latency execution of a small number of complex, sequential instruction streams. It devotes large amounts of silicon to branch prediction, out-of-order execution, and large caches so that a handful of cores can run diverse workloads quickly. A GPU takes the opposite approach: it is built for throughput rather than latency, packing thousands of simpler cores that all execute the same instruction across different pieces of data at once. This design, known as SIMT (Single Instruction, Multiple Threads), is why GPUs excel at tasks like matrix multiplication, image processing, and neural network training, where the same operation is repeated across massive datasets.

## CUDA Cores and Streaming Multiprocessors

NVIDIA GPUs are organized into Streaming Multiprocessors, or SMs, each of which contains many CUDA cores along with shared memory, registers, and specialized units for tasks like texture filtering. When a program launches a CUDA kernel, work is divided into thread blocks, and each block is scheduled onto an SM. Threads within a block can cooperate through fast shared memory, while threads in different blocks operate independently. Understanding this hierarchy, threads, warps, blocks, and grids, is essential to writing efficient CUDA code, since poor occupancy or excessive memory divergence can leave much of the GPU's compute capacity idle.

## Memory Hierarchy and Bandwidth

GPU performance is frequently limited by memory bandwidth rather than raw compute throughput. Global memory offers the largest capacity but the highest latency, while shared memory and registers are far faster but much smaller in size. Effective CUDA programming often centers on minimizing global memory traffic, coalescing memory accesses so that threads in a warp read contiguous addresses, and reusing data through shared memory whenever possible. Profiling tools such as Nsight Compute can reveal whether a kernel is compute-bound or memory-bound, which determines where optimization effort should be spent.

## Tensor Cores and Deep Learning Workloads

Modern NVIDIA GPUs include Tensor Cores, specialized units designed to accelerate the mixed precision matrix multiply-accumulate operations that dominate deep learning training and inference. Unlike general CUDA cores, Tensor Cores can perform an entire small matrix multiplication in a single operation, dramatically increasing throughput for frameworks like PyTorch and TensorFlow when mixed precision is enabled. This specialization illustrates a broader trend in GPU design: as workloads become more predictable and structured, hardware increasingly adds dedicated silicon rather than relying solely on general-purpose cores.